

Lecture Notes

Understanding Transformers

From "Attention is All You Need" to ChatGPT and Beyond

Self-Attention

Positional Encoding

Encoder & Decoder

Multi-Head Attention

Feed-Forward Layers

Training & Applications

Why Do We Need Transformers?

The problems with Recurrent Neural Networks (RNNs) that Transformers solved



Sequential processing — must wait for each word to finish before processing the next

1

Long-Range Forgetting

By the time the model reads 'mat', information about 'cat' has faded. LSTMs help but don't fully solve this.

2

Sequential = Slow

Word 6 cannot be processed until word 5 is done. No parallelism possible during training — extremely slow on GPUs.

3

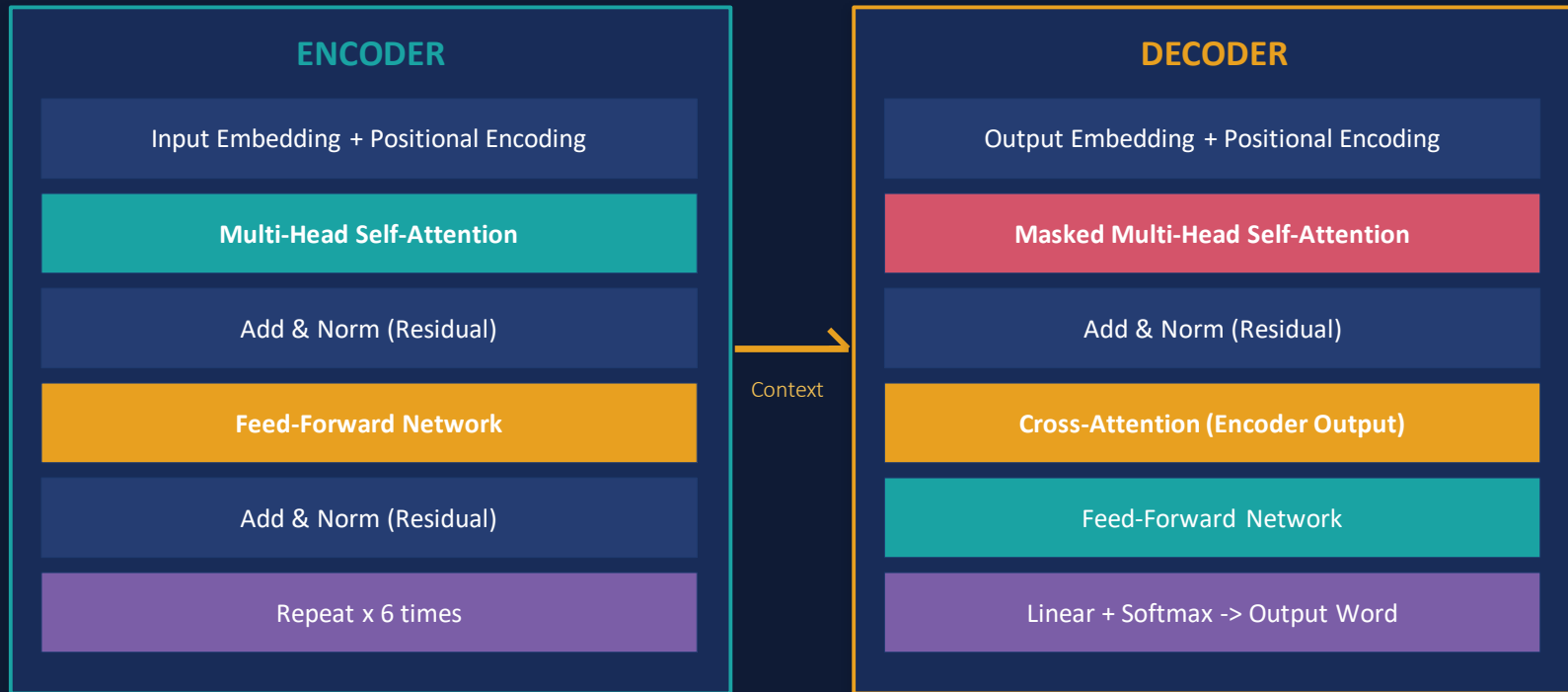
Fixed-Size Bottleneck

Entire source sentence compressed into one hidden vector. Long sentences lose critical information at this bottleneck.

Transformer Solution: Process ALL words simultaneously using attention — no sequencing, no bottleneck

The Transformer — Full Architecture

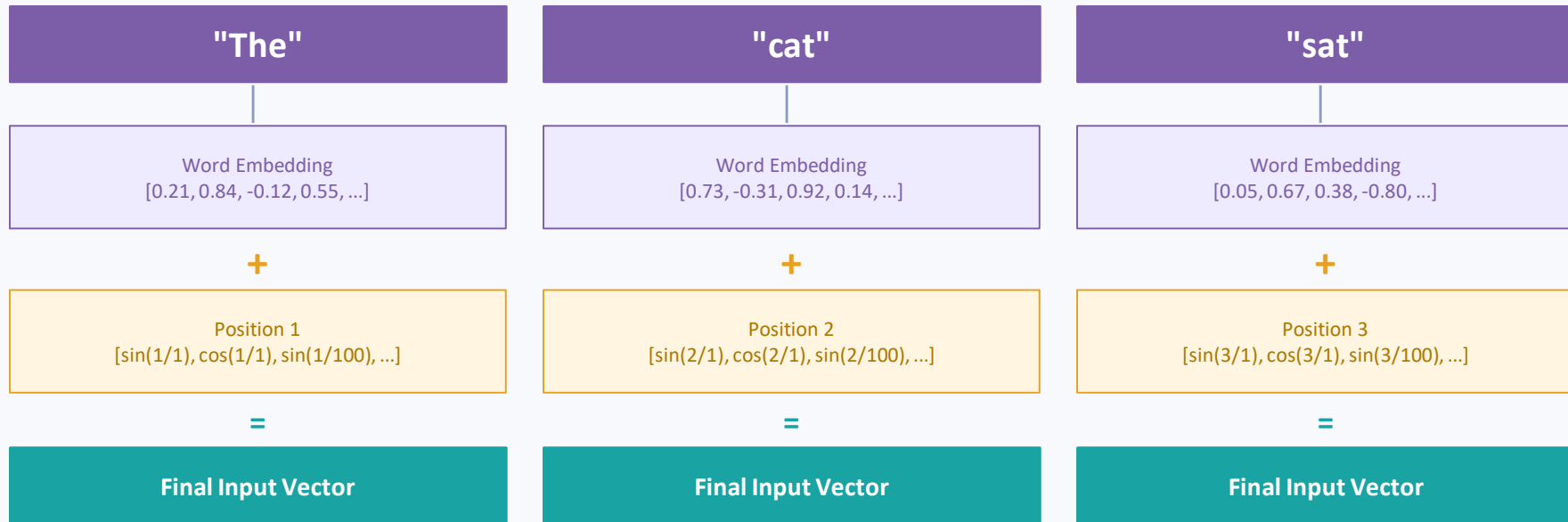
Think of it as a translation machine: Encoder reads the source, Decoder writes the target



Input Embeddings + Positional Encoding

Converting words to vectors, then injecting position information

Example sentence: "The cat sat"



Key Insight: Positional encoding uses sine/cosine waves so the model knows word ORDER — "cat sat" vs "sat cat" give different vectors

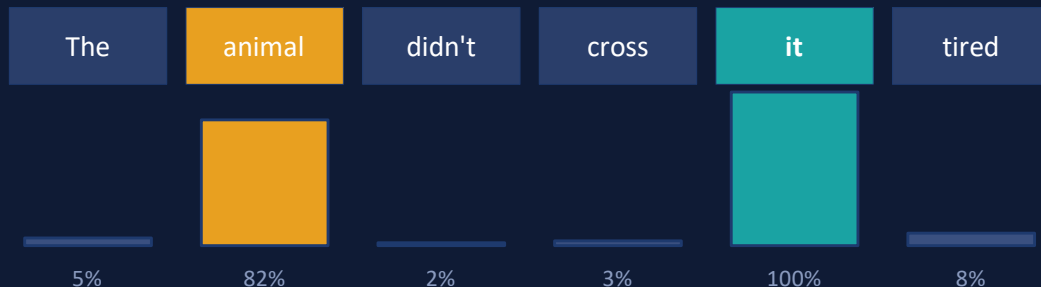
Self-Attention — The Core Idea

When reading each word, how much should it "attend" to every other word?

"The animal didn't cross the street because IT was too tired"

What does "IT" refer to? Self-attention figures this out by letting "IT" look at ALL other words.

Attention weights when processing "it":



"it" attends mostly to "animal" (82%) — correctly identifying the referent

Every word looks at every other word

Unlike RNNs, no sequential dependency. All positions computed in parallel.

Weighted relevance score

High weight = highly relevant context. Low weight = ignore. Weights sum to 1 (softmax).

Context-sensitive meaning

'Bank' near 'river' attends to water words. 'Bank' near 'money' attends to finance words.

Query, Key and Value — The QKV Mechanism

A library search analogy to build intuition



Query (Q)

Your search term
"I need books about cats"



Key (K)

Book catalogue index
"Cat anatomy, Cat history..."



Value (V)

Actual book content
you retrieve and read

How Self-Attention is Computed (step by step)

1

Create Q, K, V matrices

Multiply input embedding by 3 learned weight matrices W_Q , W_K , W_V to get Query, Key, Value for each word

2

Compute attention scores

Score = $Q \cdot K^T$ (dot product). For word 'it' and word 'animal': score = $Q_{it} \cdot K_{animal}$ = how relevant is 'animal' to 'it'?

3

Scale and normalise

Divide score by $\sqrt{\text{dimension}}$. Apply softmax to get probabilities (weights sum to 1.0). Scaling prevents vanishing gradients.

4

Weighted sum of Values

Output = $\sum(\text{weight}_i * V_i)$. High-weight words contribute more to the output. Low-weight words barely contribute.

$$\text{Formula: } \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

WORKED EXAMPLE

Self-Attention: Calculating by Hand

3 words, dimension=2 — simplified to show the mathematics clearly

Input Embeddings (dim=2)

Word	x1	x2
Thinking	1	0
Machines	0	1
Learn	1	1

For simplicity: $WQ = WK = WV = \text{Identity matrix}$

Therefore: $Q = K = V = X$ (input itself)

Score=2 means 'Learn' attends strongly to itself (diagonal = self-attention!)

Step 2: Scores = $Q * K^T$ (dot products)

	Thinking	Machines	Learn
Thinking	$1*1+0*0=1$	$1*0+0*1=0$	$1*1+0*1=1$
Machines	$0*1+1*0=0$	$0*0+1*1=1$	$0*1+1*1=1$
Learn	$1*1+1*0=1$	$1*0+1*1=1$	$1*1+1*1=2$

Step 3: Scale by $\sqrt{2}=1.41$, then Softmax

For 'Learn' row: $[1, 1, 2] / 1.41 = [0.71, 0.71, 1.41]$

Softmax $([0.71, 0.71, 1.41]) = [0.21, 0.21, 0.58]$

Step 4: Output for 'Learn' = $0.21*V_{\text{Think}} + 0.21*V_{\text{Machine}} + 0.58*V_{\text{Learn}}$

$= 0.21*[1,0] + 0.21*[0,1] + 0.58*[1,1] = [0.79, 0.79]$

'Learn' now has a richer representation that includes context from other words!

Multi-Head Attention

Run attention multiple times in parallel, each learning different relationships

Analogy: Like photographing a scene with 8 different cameras, each focused on a different aspect (colour, edges, depth, motion...) then combining all photos for a richer understanding.

Formula: $\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) * \text{WO}$
where $\text{head}_i = \text{Attention}(Q * WQ_i, K * WK_i, V * WV_i)$

Head 1

Syntactic roles
(subject-verb)

Head 2

Coreference
(pronoun-noun)

Head 3

Negation scope
(not, never, no)

Head 4

Positional
proximity

Head 5

Named entity
relations

Head 6

Semantic
similarity

Head 7

Question
focus words

Head 8

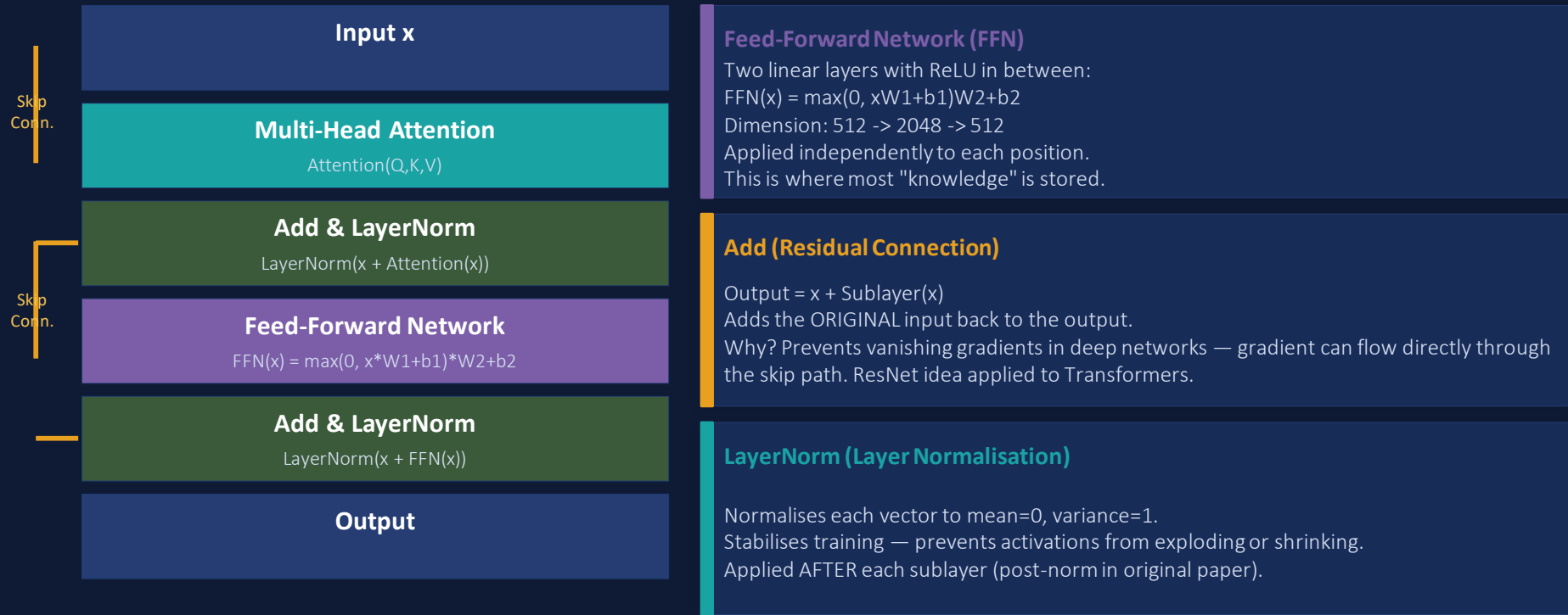
Long-range
dependencies

Concat all 8 heads -> [head1; head2; ... ; head8] -> Linear projection W_O -> Final Output

Original paper uses $h=8$ heads, $d_k = d_{\text{model}}/h = 512/8 = 64$ dimensions each

Feed-Forward Layer + Add & Norm

What happens after attention — and how residual connections stabilise training



The Encoder Stack — Building Rich Representations

N=6 identical encoder layers, each reading the FULL input sequence

Encoding the sentence: "The cat sat on the mat"

Encoder Layer 1: Each word attends to all words. "cat" attends to "The", "cat", "sat", "on", "the", "mat".

Output: 6 enriched vectors — each word now has context about all other words.

Layer 2: Layer 2 refines: detects syntactic patterns — subject (cat), verb (sat), prepositional phrase (on the mat).

Layer 3: Layer 3 refines further: semantic roles — 'cat' is the AGENT doing the sitting; 'mat' is the LOCATION.

Layer 4-5: Deeper layers: abstract representations, rarely interpretable by humans, but crucial for downstream tasks.

Layer 6: Final layer output: 6 vectors, one per word, richly encoding meaning, syntax, and context — ready for Decoder.

Bi-directional context

Each word sees ALL other words simultaneously — left AND right context, unlike RNNs

N=6 layers in paper

Original paper uses 6 encoder layers. BERT uses 12 or 24 encoder layers for deeper representations

Same structure, same size

All 6 layers are architecturally identical — only the learned weights differ after training

The Decoder — Generating Output Word by Word

Translation example: English 'The cat sat' -> French 'Le chat assis'

Step	Input to Decoder	Output Word	Masked Self-Attn: Sees	Cross-Attn: Reads
t=1	<START>	Le	Sees only <START>	Attends to all encoder outputs
t=2	Le	chat	Sees <START>, Le	Attends to all encoder outputs
t=3	Le chat	assis	Sees <START>, Le, chat	Attends to all encoder outputs
t=4	Le chat assis	<END>	Sees full output so far	Attends to all encoder outputs

Why MASKING in the Decoder?

During TRAINING, we know the full target sentence. But during INFERENCE, we predict one word at a time. Masking (setting future positions to -infinity before softmax) prevents the model from 'cheating' by looking at future words. This makes training match inference behaviour.

Cross-Attention — Connecting Encoder and Decoder

Decoder Query Q comes from decoder's own hidden state. Encoder Keys K and Values V come from encoder output. This lets the decoder 'ask questions' of the source sentence context — the bridge between encoder and decoder.

Positional Encoding — Teaching the Model Word Order

Attention alone has NO concept of order — we must add position information explicitly

Problem: "Dog bites man" and "Man bites dog" have the same words -> same embeddings -> same attention scores! The model can't tell them apart without position information.

The Sinusoidal Formula:

$$PE(pos, 2i) = \sin(pos / 10000^{(2i/d_model)})$$

$$PE(pos, 2i+1) = \cos(pos / 10000^{(2i/d_model)})$$

pos = word position, i = dimension index, d_model = 512

Unique per position

Every position gets a unique pattern of sine/cosine values — like a fingerprint for each position.

Generalises to any length

Sinusoids work for sequences longer than seen in training — unlike learned embeddings.

Sample PE values (pos=1,2,3 | dim=0,1,2):

pos	dim 0 sin	dim 1 cos	dim 2 sin
1	0.841	0.540	0.009
2	0.909	-0.416	0.019
3	0.141	-0.990	0.028

Re

PE
rel

Deterministic (no training)

Each position is a unique combination of sin/cos values across all dimensions
Fixed formula, no parameters to learn — saves training capacity for meaningful patterns.

Training the Transformer

Teacher forcing, loss function, and the Adam optimiser

1

Input pair

Source sentence: 'The cat sat'
Target sentence: 'Le chat assis'
Pair millions of such sentence pairs (e.g. WMT dataset, 4.5M pairs)

2

Teacher forcing

Feed ACTUAL target words as decoder input (not predicted words).
Speeds training dramatically — no error accumulation during training.

3

Compute loss

Cross-entropy loss between predicted word probability and actual word.
 $\text{Loss} = -\sum(y_{\text{true}} * \log(y_{\text{pred}}))$
Average over all positions.

4

Backpropagation

Compute gradient of loss with respect to ALL 65M+ parameters (weights WQ, WK, WV, WO, FFN weights, embeddings).

5

Adam optimiser

Update weights with adaptive learning rate. Warmup: lr increases for first 4000 steps, then decreases. Prevents unstable early training.

Original paper training: 8 P100 GPUs, 12 hours (base) to 3.5 days (big model), ~3.3 billion tokens, 65M parameters

Transformer Variants — BERT, GPT and T5

Same architecture, different training objectives, different use cases

BERT

2018 Google

Encoder Only

Training:

Masked Language Model (MLM): Mask 15% of tokens, predict them. Next Sentence Prediction.

Best for:

Reading comprehension, Named Entity Recognition, Sentiment Analysis, Classification

Params:

110M (base), 340M (large)

Analogy:

A student who reads an ENTIRE book to answer questions — sees all context

GPT

2018-2023 OpenAI

Decoder Only

Training:

Causal Language Model: predict the NEXT token. Simple but scales incredibly well.

Best for:

Text generation, Code, Conversation, Creative writing (ChatGPT uses GPT-4)

Params:

117M (GPT-1) -> 175B (GPT-3) -> Unknown (GPT-4)

Analogy:

A writer who composes text left-to-right — sees only past context

T5

2019 Google

Encoder + Decoder

Training:

Text-to-Text: EVERY task reformulated as text in, text out. Translation, summarisation, Q&A all identical format.

Best for:

Translation, Summarisation, Q&A, Multi-task learning. Flexible universal model.

Params:

60M (small) -> 11B (XXL)

Analogy:

A translator who reads source (encoder) and writes target (decoder) — full sequence-to-sequence

Transformers in the Real World

From machine translation to protein folding — the most impactful architecture in AI history

Natural Language Processing

- Machine Translation (Google Translate, DeepL)
- ChatGPT, Claude, Gemini — conversational AI
- Code generation: GitHub Copilot, CodeLLaMA
- Question answering and document summarisation

Computer Vision

- Vision Transformers (ViT) for image classification
- DALL-E, Stable Diffusion — image generation
- Object detection (DETR) — no anchors needed
- Medical image analysis: tumour detection

Science & Research

- AlphaFold 2: predicts protein structure — Nobel Prize 2024
- Drug discovery: molecule generation
- Climate modelling and weather prediction
- Particle physics: event classification at CERN

Multimodal AI

- GPT-4V: understands text + images together
- Speech recognition: Whisper by OpenAI
- Video understanding: narration + visual frames
- Audio generation: music, speech synthesis

Limitations and Open Research Problems

Understanding limitations is as important as understanding capabilities

Quadratic Attention Cost

Problem: Self-attention computes scores for EVERY pair of tokens. Cost = $O(n^2 * d)$. A 1000-word document has 1,000,000 attention pairs. For 10,000 tokens: 100M pairs. Impractical for very long documents.

Research: Solutions: Sparse attention (Longformer), linear attention (Linformer), FlashAttention (memory-efficient exact attention).

Lack of True Reasoning

Problem: Transformers are pattern matchers, not reasoners. They fail on novel multi-step math, causal reasoning, and tasks requiring genuine understanding not seen in training data.

Research: Active research: Chain-of-thought prompting, scratchpads, tool use, neurosymbolic AI integration.

Hallucination

Problem: Models confidently generate factually wrong information. They have no internal 'fact checker' — they predict the next likely token, not the true token. Dangerous for medical/legal applications.

Research: Retrieval-Augmented Generation (RAG), RLHF alignment, uncertainty quantification still active research.

Enormous Compute Cost

Problem: Training GPT-3 cost ~\$4.6M USD. GPT-4 estimated \$50-100M. Environmental cost: thousands of tons of CO2. Only a handful of organisations can train frontier models.

Research: Efficient architectures (Mamba, RWKV), model distillation, quantisation, mixture of experts (MoE).

What You Should Know About Transformers

1

Self-Attention is the core innovation

Every token attends to every other token simultaneously — capturing long-range dependencies without sequential processing.

4

Positional encoding gives word order

Sine/cosine patterns added to embeddings so identical words at different positions produce different vectors.

5

Encoder reads, Decoder writes

Encoder builds context vectors. Decoder generates output one token at a time using masked self-attention and cross-attention.

2

QKV is a soft retrieval mechanism

Query asks, Key matches, Value retrieves. The formula: $\text{softmax}(\text{QK}^T / \sqrt{d_k})V$ is the heart of the entire architecture.

3

Multi-head = multiple perspectives

8 heads each learning different linguistic relationships — syntax, coreference, semantics — then combined.

6

Scale + data = emergent capabilities

Larger models (GPT-4, Claude) show emergent abilities (reasoning, coding, few-shot learning) not seen at smaller scales.

REVIEW QUESTIONS

Practice Questions

Test your understanding of Transformer concepts

Conceptual

Q1: Why can't we use a standard RNN instead of a Transformer for long sequences? What specific problem does self-attention solve?

Hint: Think: sequential vs parallel, memory bottleneck, vanishing gradients

Conceptual

Q3: Why do we need positional encoding? What would happen if we removed it from the Transformer?

Hint: Consider: 'cat bit dog' vs 'dog bit cat' — would attention scores differ?

Advanced

Q5: Explain the role of residual connections (Add & Norm). What would go wrong in a 6-layer Transformer trained WITHOUT residual connections?

Hint: Consider: gradient flow through 6 layers, vanishing gradient problem

Mathematical

Q2: Given $Q=[1,0]$, $K=[[1,0],[0,1],[1,1]]$, $V=[[2,0],[0,3],[1,1]]$, compute the attention output for $d_k=2$. Show all steps.

Hint: Score = QK^T , Scale by $\sqrt{2}$, Softmax, then weighted sum of V

Design

Q4: BERT uses Encoder-only, GPT uses Decoder-only. Give a task best suited to each and explain WHY that architecture fits the task.

Hint: Think about bidirectional vs unidirectional context

Critical

Q6: A student says 'Transformers understand language just like humans do.' Do you agree or disagree? Justify with at least two specific examples.

Hint: Consider: hallucination, pattern matching vs reasoning, lack of world model